

XYZ Car Company Ltd: Car Manufacturing Software

Python

```
from abc import ABC, abstractmethod
```

1. Car Abstract Class (Product)

```
class Car(ABC):
```

```
    @abstractmethod
```

```
    def drive(self):
```

```
        """Drive the car - each type has its own behavior."""
```

```
        pass
```

2. Concrete Car Classes

```
class Saloon(Car):
```

```
    def drive(self):
```

```
        print("Driving a sleek Saloon car through the city - smooth and comfortable.")
```

```
class Pickup(Car):
```

```
    def drive(self):
```

```
        print("Driving a rugged Pickup truck on the trail - ready for heavy duty.")
```

```
class SUV(Car):
```

```
    def drive(self):
```

```
        print("Driving a powerful SUV with high clearance - excellent for all terrains.")
```

3. CarFactory Abstract Class (Creator)

```
class CarFactory(ABC):
```

```
    @abstractmethod
```

```
def create_car(self) -> Car:
    """Factory method to create specific car types."""
    pass
```

4. Concrete Factory Classes

```
class SaloonFactory(CarFactory):
```

```
    def create_car(self) -> Car:
        return Saloon()
```

```
class PickupFactory(CarFactory):
```

```
    def create_car(self) -> Car:
        return Pickup()
```

```
class SUVFactory(CarFactory):
```

```
    def create_car(self) -> Car:
        return SUV()
```

5. Client Code Demonstration

```
def start_production(factory: CarFactory, car_type: str):
```

```
    """Client code that works with abstractions only."""
```

```
    print(f"XYZ Car Company - Starting production of {car_type}...")
```

```
    car = factory.create_car()    # Factory Method in action
```

```
    car.drive()
```

```
    print("-" * 60)
```

```
if __name__ == "__main__":
```

```
print("=== XYZ Car Company - Factory Method Pattern ===\n")
```

```
production_line = [  
    (SaloonFactory(), "Saloon"),  
    (PickupFactory(), "Pickup"),  
    (SUVFactory(), "SUV")  
]
```

```
for factory, car_type in production_line:  
    start_production(factory, car_type)
```

```
print("All vehicles manufactured successfully!")
```

Notes

The above design ensures that the main program never has to hardcode specific car classes.

- Abstraction: I utilised a Car abstract class and a CarFactory interface. This guarantees that all concrete cars and factories follow the same structural contract.
- Encapsulation: Each car type—Saloon, Pickup, and SUV—is managed by its own dedicated factory. This localizes the instantiation logic, meaning adding a "Convertible" would only require one new class and one new factory without touching existing code.
- Flexibility: In the demonstration, the start_production function is completely "blind" to which car it is creating. It simply interacts with the high-level Car and CarFactory interfaces, fulfilling the core principle of the Factory Method Pattern

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley Professional.

Lott, S. F. and Phillips, D. (2021). *Python Object-Oriented Programming: Build robust and maintainable object-oriented Python applications and libraries*. 4th edn. Birmingham: Packt Publishing.